

HOSPITAL MANAGEMENT SYSTEM

Database Management System — Project Report

A Comprehensive Technical Analysis

Covering Relational Schema · Integrity Constraints · SQL Queries
RBAC · Transaction Logic · ER Diagram · Application Flow

Technology Stack: Next.js 15 · PostgreSQL · Node.js · TypeScript
Subject: Database Management Systems (DBMS)

Report Date: 2 May 2026

1. Project Overview

The Hospital Management System (HMS) is a full-stack web application designed to digitise and manage the core operational workflows of a hospital environment. It brings together four distinct user roles — Admin, Doctor, Receptionist, and Patient — under one unified platform, each with precisely scoped access rights, a dedicated dashboard, and a defined set of permitted actions. The system is underpinned by a relational PostgreSQL database that enforces referential integrity, domain constraints, and a carefully designed Role-Based Access Control (RBAC) model.

1.1 Objectives

- Provide a centralised, role-aware digital platform for managing hospital operations.
- Enforce data integrity through primary keys, foreign keys, unique constraints, and check constraints.
- Implement a full appointment lifecycle with deterministic status transitions.
- Record clinical outcomes (interactions, diagnoses, prescriptions) atomically using database transactions.
- Deliver real-time, role-specific dashboard statistics using aggregation queries.
- Protect sensitive patient data through JWT-based authentication and RBAC permission checks.

1.2 Technology Stack

The application is built as a monorepo. The frontend and API layer are part of the same Next.js 15 application. The DBMS layer is entirely PostgreSQL.

Layer	Technology	Role in System
Frontend UI	Next.js 15, React, TypeScript	Server-side rendered pages, client components
Styling	Tailwind CSS	Utility-first responsive design
API Layer	Next.js Route Handlers (REST)	Exposes HTTP endpoints consumed by the UI
Database	PostgreSQL 15+	Relational DBMS — sole source of truth
DB Driver	node-postgres (pg)	Connection pooling & parameterised queries
Auth	JWT (jsonwebtoken) + bcryptjs	Session tokens & password hashing
Middleware	Next.js Middleware (jose)	Route protection & role-based redirects
Extensions	pgcrypto (PostgreSQL)	UUID generation with gen_random_uuid()

2. Database Schema — Table-by-Table Analysis

The database consists of thirteen tables divided into two logical groups: the RBAC infrastructure tables and the domain (hospital) tables. Every primary key is a UUID generated by PostgreSQL's pgcrypto extension using gen_random_uuid(), ensuring globally unique, non-sequential identifiers that do not leak row-count information.

2.1 RBAC Infrastructure Tables

Role-Based Access Control is implemented through five tables that encode a many-to-many relationship between users, roles, and permissions. This is a textbook implementation of the NIST RBAC Standard Level 1 model.

2.1.1 Users Table

CREATE TABLE — Users		
CREATE EXTENSION IF NOT EXISTS "pgcrypto";		
CREATE TABLE Users (		
userId	UUID	PRIMARY KEY DEFAULT gen_random_uuid(),
userEmail	VARCHAR(100)	UNIQUE NOT NULL,
userPassword	VARCHAR(255)	NOT NULL,
createdAt	TIMESTAMP	DEFAULT NOW(),
updatedAt	TIMESTAMP	DEFAULT NOW()
);		

Purpose: Central identity store. Every human actor in the system — admin, doctor, receptionist, or patient — must have exactly one row in this table before any domain-level profile is created.

Column	Type	Constraint	Notes
userId	UUID	PK	Auto-generated by pgcrypto. Non-sequential to prevent enumeration attacks.
userEmail	VARCHAR(100)	UNIQUE NOT NULL	Login identifier. Case-sensitive. Uniqueness prevents duplicate accounts.
userPassword	VARCHAR(255)	NOT NULL	Stores bcrypt hash (60 chars). VARCHAR(255) accommodates future algorithm changes.
createdAt	TIMESTAMP	DEFAULT NOW()	Audit trail — records exact moment of account creation.
updatedAt	TIMESTAMP	DEFAULT NOW()	Automatically updated by trg_users_updated trigger on every UPDATE.

2.1.2 Role Table

CREATE TABLE — Role	
CREATE TABLE Role (roleId UUID PRIMARY KEY DEFAULT gen_random_uuid(), name VARCHAR(50) UNIQUE NOT NULL);	
-- Seeded values (from definedValues.sql): INSERT INTO Role (name) VALUES ('admin'), ('doctor'), ('receptionist'), ('patient');	

Purpose: Defines the finite set of system roles. The UNIQUE constraint on name prevents accidental duplicate role definitions. Current roles: admin, doctor, receptionist, patient.

2.1.3 Permission Table

CREATE TABLE — Permission	
CREATE TABLE Permission (permissionId UUID PRIMARY KEY DEFAULT gen_random_uuid(), name VARCHAR(100) UNIQUE NOT NULL);	
-- Seeded permissions: -- USER: create_user, read_user, update_user -- PATIENT: create_patient, read_patient, update_patient -- DOCTOR: create_doctor, read_doctor, update_doctor -- RECEPTIONIST: create_receptionist -- COMMON: read_own_data	

Purpose: Atomic action descriptors. Each row names a specific capability. These names are checked at the application layer by requirePermission(). The UNIQUE constraint ensures no duplicate permission strings exist.

2.1.4 UserRole Table (Junction Table)

CREATE TABLE — UserRole	
CREATE TABLE UserRole (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), userId UUID NOT NULL, roleId UUID NOT NULL);	

-- Constraints applied separately:
ALTER TABLE UserRole
ADD CONSTRAINT fk_userrole_user
FOREIGN KEY (userId) REFERENCES Users(userId) ON DELETE CASCADE,
ADD CONSTRAINT fk_userrole_role
FOREIGN KEY (roleId) REFERENCES Role(roleId) ON DELETE CASCADE,
ADD CONSTRAINT unique_user_role UNIQUE (userId, roleId);

Purpose: Implements the many-to-many relationship between Users and Roles. The composite UNIQUE(userId, roleId) constraint prevents a user from being assigned the same role twice. ON DELETE CASCADE on userId means if a user is deleted, all their role assignments are automatically removed — no orphan records.

Important design note: The application logic in assignRole() first DELETES all existing UserRole rows for a userId before inserting a new one. This enforces a single-role-per-user policy at the application level, even though the schema technically allows multiple roles.

2.1.5 RolePermission Table (Junction Table)

CREATE TABLE — RolePermission			
CREATE TABLE RolePermission (			
id	UUID	PRIMARY KEY	DEFAULT gen_random_uuid(),
roleId	UUID	NOT NULL,	
permissionId	UUID	NOT NULL	
);			
ALTER TABLE RolePermission			
ADD CONSTRAINT fk_roleperm_role			
FOREIGN KEY (roleId) REFERENCES Role(roleId) ON DELETE CASCADE,			
ADD CONSTRAINT fk_roleperm_perm			
FOREIGN KEY (permissionId) REFERENCES Permission(permissionId) ON DELETE CASCADE,			
ADD CONSTRAINT unique_role_permission UNIQUE (roleId, permissionId);			

Purpose: Binds roles to their permitted actions. ON DELETE CASCADE on both foreign keys ensures that if a role or permission is deleted, all associated assignments are cleaned up automatically, maintaining referential integrity throughout.

2.1.6 Permission Assignment Matrix

The definedValues.sql script seeds the permission assignments using SQL JOIN-based INSERT...SELECT patterns. The following table summarises the resulting access matrix:

Permission	Admin	Receptionist	Doctor	Patient
create_user	✓ Yes	✓ Yes	✗ No	✗ No
read_user	✓ Yes	✓ Yes	✗ No	✗ No
update_user	✓ Yes	✗ No	✗ No	✗ No
create_patient	✓ Yes	✓ Yes	✗ No	✗ No
read_patient	✓ Yes	✓ Yes	✓ Yes	✗ No
update_patient	✓ Yes	✗ No	✗ No	✗ No
create_doctor	✓ Yes	✗ No	✗ No	✗ No
read_doctor	✓ Yes	✗ No	✓ Yes	✗ No
update_doctor	✓ Yes	✗ No	✓ Yes	✗ No
create_receptionist	✓ Yes	✗ No	✗ No	✗ No
read_own_data	✓ Yes	✗ No	✓ Yes	✓ Yes

The Admin is seeded using a CROSS JOIN (giving all permissions), while other roles are seeded using an INNER JOIN with a specific IN() list — a precise, efficient pattern that avoids hardcoding UUID values.

2.2 Domain Tables

The domain tables model the hospital's operational entities. They are linked to the Users table, forming a hub-and-spoke identity model.

2.2.1 Patient Table

CREATE TABLE — Patient		
CREATE TABLE Patient (patientId UUID PRIMARY KEY DEFAULT gen_random_uuid(), userId UUID UNIQUE, patientNumber BIGSERIAL UNIQUE, fullname VARCHAR(100) NOT NULL, address TEXT, gender VARCHAR(20), createdAt TIMESTAMP DEFAULT NOW(), updatedAt TIMESTAMP DEFAULT NOW());		
ALTER TABLE Patient ADD CONSTRAINT fk_patient_user FOREIGN KEY (userId) REFERENCES Users(userId) ON DELETE SET NULL;		

Purpose: Holds demographic and profile data for patients. Noteworthy design decisions:

- patientNumber uses BIGSERIAL (auto-incrementing integer) to provide a human-friendly, sequential patient identifier for hospital staff, separate from the UUID primary key.
- userId is UNIQUE — enforcing that one user account maps to at most one patient profile (one-to-one relationship).
- ON DELETE SET NULL on the foreign key means: if the user account is deleted, the patient record is not cascaded away. Instead, userId is set to NULL, preserving the patient's medical history even after account removal. This is a critical data-integrity decision in a medical context.
- address and gender are nullable (no NOT NULL constraint) — they are optional profile fields completed post-registration.

2.2.2 Doctor Table

CREATE TABLE — Doctor		
CREATE TABLE Doctor (doctorId UUID PRIMARY KEY DEFAULT gen_random_uuid(), userId UUID UNIQUE NOT NULL, fullname VARCHAR(100) NOT NULL,		

specialization	VARCHAR(100),
status	VARCHAR(50),
createdAt	TIMESTAMP DEFAULT NOW(),
updatedAt	TIMESTAMP DEFAULT NOW()
);	
ALTER TABLE Doctor	
ADD CONSTRAINT fk_doctor_user	
FOREIGN KEY (userId) REFERENCES Users(userId) ON DELETE CASCADE;	

Purpose: Stores doctor-specific professional data. ON DELETE CASCADE here means that if a doctor's user account is deleted, the entire doctor profile is also deleted. This is in contrast to patients (where SET NULL is used). The reasoning is medical record preservation: patient histories should outlive account deletion, but a doctor's availability profile is meaningless without the account.

2.2.3 Receptionist Table

CREATE TABLE — Receptionist		
CREATE TABLE Receptionist (		
receptionistId	UUID	PRIMARY KEY DEFAULT gen_random_uuid(),
userId	UUID	UNIQUE NOT NULL,
fullname	VARCHAR(100)	NOT NULL,
createdAt	TIMESTAMP	DEFAULT NOW(),
updatedAt	TIMESTAMP	DEFAULT NOW()
);		
ALTER TABLE Receptionist		
ADD CONSTRAINT fk_receptionist_user		
FOREIGN KEY (userId) REFERENCES Users(userId) ON DELETE CASCADE;		

Purpose: Identifies staff members who perform front-desk operations. Like doctors, ON DELETE CASCADE is applied since receptionists are administrative staff without independent medical record significance.

2.2.4 Appointment Table

Inferred Schema — Appointment (from application code)		
CREATE TABLE Appointment (		
appointmentId	UUID	PRIMARY KEY DEFAULT gen_random_uuid(),
patientId	UUID	NOT NULL,

doctorId	UUID	NOT NULL,
receptionistId	UUID,	
requestId	UUID,	
starttime	TIMESTAMP	NOT NULL,
endtime	TIMESTAMP,	
type	VARCHAR(50)	DEFAULT 'walk-in',
status	VARCHAR(50)	DEFAULT 'scheduled'
);		
-- Foreign Keys (inferred from application layer):		
-- FOREIGN KEY (patientId) REFERENCES Patient(patientId)		
-- FOREIGN KEY (doctorId) REFERENCES Doctor(doctorId)		
-- FOREIGN KEY (receptionistId) REFERENCES Receptionist(receptionistId)		

Purpose: The central operational table. Each row represents a scheduled visit between a patient and a doctor. The status column drives the entire appointment lifecycle state machine (described in detail in Section 4).

Column	Type	Description
appointmentId	UUID PK	Unique identifier generated at insert time.
patientId	UUID FK	References Patient.patientId. Links appointment to the patient.
doctorId	UUID FK	References Doctor.doctorId. Identifies the attending doctor.
receptionistId	UUID FK	References Receptionist. Nullable for walk-ins created without a receptionist.
requestId	UUID	Optional link to an appointment request (UI feature, not yet backed by a DB table).
starttime	TIMESTAMP	Scheduled start. Conflicts checked at this column during booking.
endtime	TIMESTAMP	Calculated as starttime + 30 minutes at insert. Updated to NOW() upon completion.
type	VARCHAR(50)	Defaults to 'walk-in'. Supports extensibility for other appointment types.
status	VARCHAR(50)	Lifecycle status: 'scheduled' → 'checked_in' → 'in_progress' → 'completed' / 'no_show'.

2.2.5 Interaction Table

Inferred Schema — Interaction		
CREATE TABLE Interaction (		
interactionId	UUID	PRIMARY KEY DEFAULT gen_random_uuid(),
appointmentId	UUID	NOT NULL UNIQUE,

notes	TEXT,
createdAt	TIMESTAMP DEFAULT NOW()
);	
-- FOREIGN KEY (appointmentId) REFERENCES Appointment(appointmentId)	
-- The UNIQUE constraint enforces exactly one interaction per appointment.	

Purpose: Serves as the root of the clinical record tree for a single appointment visit. One interaction exists per completed appointment. Child tables (Diagnosis and Prescription) hang off this table, not directly off Appointment, creating a clean clinical data hierarchy.

2.2.6 Diagnosis Table

Inferred Schema — Diagnosis	
CREATE TABLE Diagnosis (	
diagnosisId	UUID PRIMARY KEY DEFAULT gen_random_uuid(),
interactionId	UUID NOT NULL,
description	TEXT NOT NULL
);	
-- FOREIGN KEY (interactionId) REFERENCES Interaction(interactionId)	

Purpose: Stores one or more diagnostic findings for an interaction. The one-to-many relationship (one Interaction → many Diagnoses) accommodates multi-diagnosis visits, which is common in medical practice.

2.2.7 Prescription Table

Inferred Schema — Prescription	
CREATE TABLE Prescription (	
prescriptionId	UUID PRIMARY KEY DEFAULT gen_random_uuid(),
interactionId	UUID NOT NULL,
medicine	VARCHAR(200) NOT NULL,
frequency	VARCHAR(100),
duration	VARCHAR(100),
instructions	TEXT
);	
-- FOREIGN KEY (interactionId) REFERENCES Interaction(interactionId)	

Purpose: Records medicines prescribed during a visit. Each row is one medicine line item. The one-to-many relationship allows prescribing multiple medications per visit. Columns capture the full clinical metadata: what medicine, how often, for how long, and any special instructions.

3. Entity-Relationship Diagram & Relational Model

The following ER Diagram was provided as part of the project submission and illustrates the complete relational schema. Below it, a full written analysis of every relationship, cardinality, and key explains the logical structure of the database.

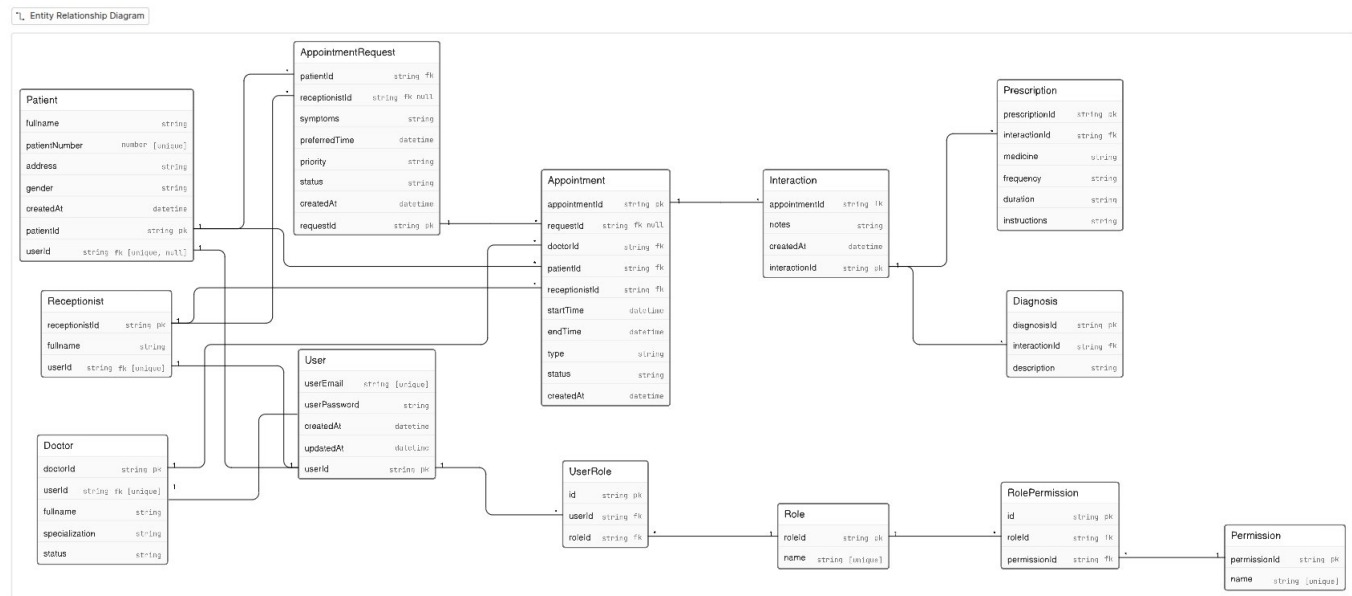


Figure 1: Entity-Relationship Diagram — Hospital Management System

3.1 Complete Relationship Catalogue

Entity A	Cardinality	Entity B	FK Column	On Delete Behaviour
Users	1 : 0..1	Patient	Patient.userId	SET NULL — preserves medical history
Users	1 : 0..1	Doctor	Doctor.userId	CASCADE — doctor profile is meaningless without account
Users	1 : 0..1	Receptionist	Receptionist.userId	CASCADE — same reasoning as Doctor
Users	1 : N	UserRole	UserRole.userId	CASCADE — removes all role bindings
Role	1 : N	UserRole	UserRole.roleId	CASCADE — removes all user-role links

Role	1 : N	RolePermission	RolePermission.roleId	CASCADE — removes permission assignments
Permission	1 : N	RolePermission	RolePermission.permissionId	CASCADE — removes role-permission links
Patient	1 : N	Appointment	Appointment.patientId	RESTRICT / default — appointments preserved
Doctor	1 : N	Appointment	Appointment.doctorId	RESTRICT / default — appointments preserved
Receptionist	1 : N	Appointment	Appointment.receptionistId	NULLABLE — receptionist is optional
Appointment	1 : 0..1	Interaction	Interaction.appointmentId	CASCADE — clinical record lives and dies with appointment
Interaction	1 : N	Diagnosis	Diagnosis.interactionId	CASCADE — diagnoses deleted with interaction
Interaction	1 : N	Prescription	Prescription.interactionId	CASCADE — prescriptions deleted with interaction

3.2 Normalisation Analysis

The schema demonstrates adherence to the third normal form (3NF) throughout:

- First Normal Form (1NF): Every column contains atomic values. No repeating groups exist. For example, diagnoses are stored as individual rows in the Diagnosis table rather than as a comma-separated string in Appointment.
- Second Normal Form (2NF): All non-key attributes are fully functionally dependent on the entire primary key. In junction tables (UserRole, RolePermission), the only non-key is the auto-generated id, which is fully dependent on the PK.
- Third Normal Form (3NF): No transitive dependencies. Patient demographic data (address, gender) is stored in Patient, not in Appointment. Doctor specialization is stored in Doctor, not duplicated in Appointment. Clinical data (notes, diagnoses, prescriptions) is separated into Interaction, Diagnosis, and Prescription tables.

4. SQL Queries — In-Depth Analysis

Every SQL query used in the system is presented below with a complete explanation of its logic, the joins employed, any subqueries, and the reason for each design choice.

4.1 Authentication & User Management Queries

4.1.1 User Lookup by Email (Login)

auth.service.ts → login()

```
SELECT * FROM users WHERE useremail = $1
```

Analysis: A simple equality filter on useremail. Because the email column carries a UNIQUE constraint, PostgreSQL automatically creates a B-Tree index on it, making this lookup $O(\log n)$ rather than a full table scan. The parameterised \$1 placeholder prevents SQL injection entirely. After retrieval, the application calls bcrypt.compare() to verify the submitted plaintext password against the stored hash.

4.1.2 Get User Roles (Post-Login Token Building)

user.service.ts → getUserRoles()

```
SELECT r.name
FROM role r
JOIN userrole ur ON r.roleid = ur.roleid
WHERE ur.userid = $1
```

Analysis: An INNER JOIN between role and userrole. The join condition r.roleid = ur.roleid traverses the many-to-many junction table. Since the application enforces one role per user (by deleting all existing rows before inserting), this query returns exactly one row in practice. The role name is then embedded in the JWT token payload, enabling the middleware to make routing decisions without a database call.

4.1.3 Assign Role (Admin Action)

user.service.ts → assignRole()

```
-- Step 1: Resolve role name to UUID
SELECT roleid FROM role WHERE name = $1

-- Step 2: Remove any existing role assignments
```

```
DELETE FROM userrole WHERE userid = $1

-- Step 3: Insert new role assignment
INSERT INTO userrole (userid, roleid) VALUES ($1, $2)
```

Analysis: A three-step atomic sequence. The first query translates a human-readable role name into its UUID, decoupling the application from UUIDs. The DELETE ensures the single-role invariant at the data layer. The final INSERT creates the new binding. Although these three queries are not wrapped in an explicit transaction here, the short sequential execution makes partial failure extremely unlikely. The more correct pattern would be to wrap in BEGIN/COMMIT for production hardening.

4.2 Admin Dashboard — Multi-Table Aggregation Query

user.service.ts → getUsersForAdminDashboard()

```
SELECT
  u.userid,
  u.useremail,
  r.name          AS role,
  COALESCE(d.fullname, rec.fullname, p.fullname) AS fullname,
  p.address,
  p.gender,
  p.patientid,
  d.specialization,
  d.doctorid,
  rec.receptionistid
FROM users u
  JOIN userrole ur  ON u.userid = ur.userid
  JOIN role r      ON ur.roleid = r.roleid
  LEFT JOIN doctor d      ON d.userid = u.userid
  LEFT JOIN receptionist rec ON rec.userid = u.userid
  LEFT JOIN patient p      ON p.userid = u.userid
ORDER BY u.createdat DESC
```

Analysis: This is the most structurally complex query in the system. It must retrieve all user accounts while joining to whichever role-specific profile table applies, since a given user will only have a row in one of the three domain tables. The design uses:

- INNER JOIN to userrole and role — guarantees only users with assigned roles appear. A user without a role is an incomplete record.
- LEFT JOIN to doctor, receptionist, patient — each of these may or may not have a row for the given user, so LEFT JOIN correctly returns NULL columns for non-matching role tables instead of dropping the user row.

- `COALESCE(d.fullname, rec.fullname, p.fullname)` — evaluates left-to-right and returns the first non-NULL value. Since only one of the three tables will have a matching row, this pattern cleanly extracts the correct fullname regardless of which role the user has.
- `ORDER BY u.createdat DESC` — returns newest accounts first, which is the expected default for an admin management view.

4.3 Appointment Queries

4.3.1 Conflict Check Before Booking

appointment.service.ts → createAppointment() — Conflict Check
<code>SELECT 1</code>
<code>FROM appointment</code>
<code>WHERE doctorid = \$1</code>
<code>AND starttime = \$2</code>
<code>AND status != 'cancelled'</code>

Analysis: A guard query executed before every INSERT. It checks whether the specific doctor already has a non-cancelled appointment at the exact requested starttime. `SELECT 1` is used rather than `SELECT *` because we only care about existence, not the data. If `conflict.rows.length > 0`, the service throws an error and the insert is blocked. The `status != 'cancelled'` exclusion is essential — cancelled slots should be re-bookable. This query acts as a soft unique constraint since no formal `UNIQUE(doctorid, starttime)` database constraint exists; the check is application-enforced.

4.3.2 Create Appointment

appointment.service.ts → createAppointment() — INSERT
<code>INSERT INTO appointment (</code>
<code> patientid, doctorid, receptionistid, requestid,</code>
<code> starttime, endtime, type, status</code>
<code>) VALUES (\$1, \$2, \$3, \$4, \$5, \$6, \$7, 'scheduled')</code>
<code>RETURNING *</code>

Analysis: A straightforward parameterised INSERT. `RETURNING *` is used so the application can immediately return the newly created appointmentId to the client without a follow-up SELECT. The endtime is calculated in JavaScript as `starttime + 30 minutes` before the query runs, since PostgreSQL interval arithmetic is not used here. status is hardcoded to 'scheduled' — the initial state of every appointment — ensuring that no appointment can be born in any other state.

4.3.3 Get All Appointments (Receptionist View)

appointment.service.ts → getAllAppointments()	
SELECT	
a.appointmentid	AS id,
p.fullname	AS patientname,
d.fullname	AS doctorname,
a.starttime	AS scheduledat,
a.status	
FROM appointment a	
JOIN patient p ON a.patientid = p.patientid	
JOIN doctor d ON a.doctorid = d.doctorid	
ORDER BY a.starttime DESC	

Analysis: A two-level INNER JOIN query that denormalises the appointment record for display purposes. Raw UUIDs (patientid, doctorid) stored in the appointment table are resolved to human-readable names by joining to the Patient and Doctor tables. INNER JOIN is appropriate because every appointment should have a valid patient and doctor — an orphaned appointment is a data integrity violation. ORDER BY starttime DESC shows the most recent/upcoming appointments first.

4.3.4 Doctor Availability Search

doctor.service.ts → searchDoctorsForAppointment()	
SELECT	
d.doctorid,	
d.fullname,	
d.specialization,	
CASE	
WHEN a.appointmentid IS NOT NULL THEN TRUE	
ELSE FALSE	
END AS is_busy	
FROM doctor d	
LEFT JOIN Appointment a	
ON d.doctorid = a.doctorid	
AND a.starttime = \$1::TIMESTAMP	
AND a.status NOT IN ('cancelled', 'rejected')	
WHERE 1=1	
[AND d.specialization = \$2] -- appended dynamically if filter provided	

Analysis: This is a conditional LEFT JOIN availability query. The left join ensures all doctors appear in results, not just busy ones. The join condition includes three predicates: the doctor match (d.doctorid = a.doctorid), the exact timeslot check (a.starttime = \$1), and a status filter that excludes cancelled/rejected appointments (so cancelled slots are treated as available). The CASE expression then produces a boolean is_busy flag: if any appointment row matched the join, appointmentid will

be non-null, indicating the doctor is busy; otherwise it is NULL, indicating availability. The WHERE 1=1 is a safe dynamic SQL pattern — the specialization filter (\$2) is appended only when the UI sends it, making the query adaptable without separate code paths.

4.3.5 Appointment Lifecycle Status Updates

appointment.service.ts — Status Transition Queries
-- CHECK-IN (Receptionist: scheduled → checked_in)
UPDATE appointment SET status = 'checked_in'
WHERE appointmentid = \$1 RETURNING *
-- START (Doctor: checked_in → in_progress, actualise starttime)
UPDATE appointment SET status = 'in_progress', starttime = NOW()
WHERE appointmentid = \$1 RETURNING *
-- COMPLETE (Doctor: in_progress → completed, record real endtime)
UPDATE appointment SET status = 'completed', endtime = NOW()
WHERE appointmentid = \$1 RETURNING *
-- NO-SHOW (Receptionist: scheduled → no_show)
UPDATE appointment SET status = 'no_show'
WHERE appointmentid = \$1 RETURNING *

Analysis: Each transition is a simple UPDATE with RETURNING * to confirm the new state. Before each UPDATE, the service fetches the current appointment (SELECT * FROM appointment WHERE appointmentid = \$1) and validates that the current status is the legal predecessor for the requested transition. This is a state-machine guard pattern implemented at the application layer. If the status is wrong, an error is thrown immediately and the UPDATE never executes. RETURNING * allows the API to respond with the updated appointment without an additional round-trip.

4.4 Clinical Data Queries (Transaction Block)

interaction.service.ts → saveClinicalData() — ACID Transaction
BEGIN;
-- Step 1: Create the Interaction header record
INSERT INTO interaction (appointmentid, notes)
VALUES (\$1, \$2)
RETURNING interactionid;
-- Step 2: Batch insert all diagnoses (N rows in one query)
INSERT INTO diagnosis (interactionid, description)
VALUES (\$1, \$2), (\$1, \$3), (\$1, \$4) -- dynamic based on count
-- Step 3: Batch insert all prescriptions (N rows in one query)
INSERT INTO prescription (interactionid, medicine, frequency, duration,
instructions)
VALUES (\$1, \$2, \$3, \$4, \$5), (\$1, \$6, \$7, \$8, \$9) -- dynamic
-- Step 4: Mark appointment as completed
UPDATE appointment SET status = 'completed' WHERE appointmentid = \$1;
COMMIT;
-- On any error: ROLLBACK automatically reverts all four steps

Analysis: This is the only query group in the system that uses an explicit database transaction with BEGIN, COMMIT, and ROLLBACK. This is the correct and essential use of transactions. The four operations form an atomic clinical unit:

1. If the interaction row inserts but the diagnosis batch fails, the interaction would exist with no diagnoses — an invalid, partial clinical record. The transaction prevents this.
2. If prescriptions fail after diagnoses succeed, we would have an inconsistent record. ROLLBACK undoes all work.
3. The appointment status update is bundled into the same transaction — ensuring that status becomes 'completed' only when all clinical data is safely committed. There is no state where the appointment is marked complete but the clinical notes are absent.

Batch INSERT pattern analysis: The diagnoses and prescriptions are inserted using dynamically constructed multi-row VALUE clauses (N tuples in one statement). This is significantly more efficient than N individual INSERT statements because it reduces round-trips to the database from N to 1, and PostgreSQL can optimise a single multi-row INSERT far better than N sequential ones. The parameterisation is built by mapping array indices to positional parameters (\$1, \$2, \$3...), correctly avoiding SQL injection while still being dynamic.

4.5 Clinical Data Retrieval — Aggregation with json_agg

interaction.service.ts → getInteractionByAppointment()

```
SELECT
  i.interactionid,
  i.notes,
  json_agg(DISTINCT jsonb_build_object(
    'description', d.description
  )) FILTER (WHERE d.description IS NOT NULL) AS diagnoses,
  json_agg(DISTINCT jsonb_build_object(
    'medicine',      p.medicine,
    'frequency',     p.frequency,
    'duration',      p.duration,
    'instructions',  p.instructions
  )) FILTER (WHERE p.medicine IS NOT NULL) AS prescriptions
FROM interaction i
  LEFT JOIN diagnosis  d ON i.interactionid = d.interactionid
  LEFT JOIN prescription p ON i.interactionid = p.interactionid
WHERE i.appointmentid = $1
GROUP BY i.interactionid, i.notes
```

Analysis: This is the most sophisticated read query in the system. It retrieves a complete clinical interaction record — including all diagnoses and all prescriptions — in a single database round-trip using PostgreSQL's `json_agg` aggregate function. Key points:

- **LEFT JOIN** to both diagnosis and prescription: An interaction may have zero diagnoses or zero prescriptions (though unusual). **LEFT JOIN** ensures the interaction row itself is always returned even if there are no child rows.
- **json_agg(DISTINCT ...)**: Aggregates multiple rows from the joined tables into a single JSON array per interaction. **DISTINCT** prevents duplicate entries that could arise from the cross-join effect when both diagnosis and prescription tables have multiple rows.
- **jsonb_build_object()**: Constructs a structured JSON object per row from multiple columns, returning it as a typed JSONB value. The application receives a ready-to-use array of objects rather than flat rows.
- **FILTER (WHERE ... IS NOT NULL)**: Essential guard clause. If a **LEFT JOIN** produces a **NULL** row (no diagnoses/prescriptions exist), **json_agg** would aggregate **NULL** into a **[null]** array. The **FILTER** clause makes it return **NULL** instead, which the application handles as an empty list.
- **GROUP BY i.interactionid, i.notes**: Required because **json_agg** is an aggregate function. Since we are collapsing multiple rows from the joined tables into single interaction rows, the **GROUP BY** must include all non-aggregated columns selected.

4.6 Dashboard Queries — Aggregation & Subqueries

4.6.1 Doctor Dashboard Statistics

dashboard.service.ts → getDoctorDashboardData() — Stats Query

```
SELECT
  COUNT(*) FILTER (WHERE starttime::date = CURRENT_DATE)
  AS today_appointments,
  COUNT(*) FILTER (WHERE status = 'completed')
  AS patients_seen,
  COUNT(*) FILTER (WHERE status = 'in_progress')
  AS pending_records
FROM appointment
WHERE doctorid = (SELECT doctorid FROM doctor WHERE userid = $1)
```

Analysis: A correlated scalar subquery combined with conditional COUNT aggregates. The subquery (SELECT doctorid FROM doctor WHERE userid = \$1) resolves the logged-in user's UUID to their doctor profile UUID in a single inline step, avoiding a separate query. The three FILTER clauses on COUNT(*) produce three separate statistics in a single table scan — this is more efficient than three separate COUNT queries. starttime::date casts the timestamp to a date for day-level comparison with CURRENT_DATE.

4.6.2 Doctor's Next Patient Query

dashboard.service.ts → getDoctorDashboardData() — Next Patient

```
SELECT
  a.appointmentid,
  a.starttime,
  p.fullname AS patientname,
  p.gender,
  a.type,
  a.status
FROM appointment a
JOIN patient p ON a.patientid = p.patientid
WHERE
  a.doctorid = (SELECT doctorid FROM doctor WHERE userid = $1)
  AND a.status IN ('checked_in', 'in_progress')
ORDER BY
  CASE WHEN a.status = 'in_progress' THEN 0 ELSE 1 END,
  a.starttime ASC
LIMIT 1
```

Analysis: A priority-ordered single-record retrieval query. The ORDER BY uses a CASE expression as a computed sort key — it assigns rank 0 to in_progress appointments and rank 1 to checked_in ones. This ensures that if a patient is actively in consultation (in_progress), they are shown as the 'current' patient. After that, checked_in patients are ordered by appointment time. LIMIT 1 returns only the single most urgent record for the dashboard widget.

4.6.3 Patient Activity Feed (UNION ALL)

dashboard.service.ts → getPatientDashboardData() — Activity Feed	
SELECT * FROM (	
-- Branch 1: Appointment events	
SELECT	
'Appointment' AS type,	
a.starttime AS date,	
a.status AS status,	
'Appointment with ' d.fullname AS title,	
a.appointmentid::text AS id	
FROM appointment a	
JOIN doctor d ON a.doctorid = d.doctorid	
WHERE a.patientid = (SELECT patientid FROM patient WHERE userid = \$1)	
UNION ALL	
-- Branch 2: Clinical record events	
SELECT	
'Record' AS type,	
i.createdat AS date,	
'info' AS status,	
'Medical record updated' AS title,	
i.interactionid::text AS id	
FROM interaction i	
JOIN appointment a ON i.appointmentid = a.appointmentid	
WHERE a.patientid = (SELECT patientid FROM patient WHERE userid = \$1)	
) AS combined_activity	
ORDER BY date DESC	
LIMIT 5	

Analysis: A UNION ALL query that merges two semantically different event types into a single chronological feed. UNION ALL (not UNION) is used because there is no possibility of duplicate rows across the two branches — an appointment event and a record event will never be identical. UNION would add unnecessary overhead by performing deduplication. Each branch selects a consistent set of columns with matching aliases (type, date, status, title, id), making them union-compatible. The outer ORDER BY date DESC LIMIT 5 then sorts the combined result set and returns only the 5 most recent items. The ::text casts on UUIDs ensure type consistency across both SELECT branches.

4.7 Permission Resolution Query

permission.ts → getUserRoleAndPermissions()
-- Step 1: Get the user's role
SELECT r.name
FROM role r
JOIN userrole ur ON r.roleid = ur.roleid
WHERE ur.userid = \$1
-- Step 2: Get all permissions for that role
SELECT p.name
FROM permission p
JOIN rolepermission rp ON p.permissionid = rp.permissionid
JOIN role r ON rp.roleid = r.roleid
WHERE r.name = \$1 -- the role name from Step 1

Analysis: Two sequential JOIN queries form the RBAC enforcement chain. The first query traverses Users → UserRole → Role to identify what role the logged-in user has. The second traverses Role → RolePermission → Permission to collect all permission strings for that role. The returned array of permission names (e.g. ['read_patient', 'create_appointment', ...]) is checked synchronously by requirePermission() before every protected API operation executes. This two-step resolution is called on every authenticated API request, making it the most frequently executed query group in the system.

5. Database Triggers

The system implements an automatic timestamp maintenance mechanism using a PostgreSQL trigger function that fires on UPDATE events across all audited tables.

triggers.sql — set_updated_at() Function & Triggers

```
-- Shared trigger function
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updatedAt = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Applied to four tables:
CREATE TRIGGER trg_users_updated
    BEFORE UPDATE ON Users
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();

CREATE TRIGGER trg_patient_updated
    BEFORE UPDATE ON Patient
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();

CREATE TRIGGER trg_doctor_updated
    BEFORE UPDATE ON Doctor
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();

CREATE TRIGGER trg_receptionist_updated
    BEFORE UPDATE ON Receptionist
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

Analysis:

- **Trigger type: BEFORE UPDATE** — the trigger fires before the UPDATE statement completes, allowing it to modify the NEW row before it is committed to disk.
- **FOR EACH ROW** — the trigger fires once per modified row, making it correct for both single-row and batch updates.
- **NEW.updatedAt = NOW()** — sets the updatedAt column to the current database server timestamp. This is database-enforced, not application-enforced, meaning it is impossible for an UPDATE to leave updatedAt stale, even if a developer forgets to include it in the SET clause.
- **CREATE OR REPLACE** — the function is idempotent. Re-running the setup script will replace the existing function rather than erroring.

- `LANGUAGE plpgsql` — uses PostgreSQL's procedural language for the function body.

Scope: The triggers are applied to Users, Patient, Doctor, and Receptionist — the tables that store profile data subject to user-initiated edits. Appointment, Interaction, Diagnosis, and Prescription do not have `updatedAt` triggers because those tables are append-only by design (appointments transition by status update, clinical data is written once).

6. Data Integrity Constraints

The system enforces data integrity at multiple layers: the database schema layer, the application service layer, and the API route layer. This defence-in-depth approach ensures that invalid data cannot be stored, even if the UI or API is called directly.

6.1 Database-Level Constraints

Table	Column	Constraint	Effect
Users	userId	PRIMARY KEY	Guarantees row uniqueness; auto-generates UUID.
Users	userEmail	UNIQUE NOT NULL	Prevents duplicate accounts; requires email at registration.
Users	userPassword	NOT NULL	No account can exist without a password hash.
Role	name	UNIQUE NOT NULL	No duplicate role names.
Permission	name	UNIQUE NOT NULL	No duplicate permission strings.
UserRole	(userId, roleId)	UNIQUE	Prevents a user from being assigned the same role twice.
RolePermission	(roleId, permId)	UNIQUE	Prevents duplicate permission assignments.
Patient	userId	UNIQUE	One patient profile per user account.
Patient	patientNumber	UNIQUE (BIGSERIAL)	Sequential unique patient ID for human identification.
Doctor	userId	UNIQUE NOT NULL	One doctor profile per user account.
Receptionist	userId	UNIQUE NOT NULL	One receptionist profile per user account.

6.2 Application-Level Constraints

- Email uniqueness check before insert: `auth.service.ts` calls `findByEmail()` before creating a user and throws "User already exists" if a record is found. This provides a user-friendly error message ahead of the database UNIQUE constraint violation.
- Appointment conflict check: Before inserting an appointment, a `SELECT` is run to verify the doctor has no overlapping non-cancelled appointment at the same starttime. The database lacks a `UNIQUE(doctorid, starttime)` constraint, so this is the sole enforcement mechanism for booking conflicts.

- Status transition validation: Every appointment status change (checkin, start, complete, no-show) first fetches the current appointment and explicitly validates the predecessor status. Illegal transitions (e.g., completing a 'scheduled' appointment) throw a descriptive error.
- Role permission enforcement: Every protected API endpoint calls `requirePermission()` before executing business logic. The permission is resolved from the database, not from the JWT, preventing privilege escalation by manipulating the token.
- Ownership checks on appointment detail: The `GET /api/appointments/[id]` endpoint verifies that the requesting user is the doctor or patient for that appointment, or is a receptionist. No user can read another user's appointment data.

7. Authentication & Security Architecture

The system implements a stateless JWT-based authentication mechanism layered over the database RBAC model. Every request goes through an authentication pipeline before any database query is executed.

7.1 Registration Flow

1. User submits email + password + fullname via POST /api/auth/register.
2. auth.service.ts calls userService.findByEmail() — if a record exists, registration is rejected.
3. bcrypt.genSalt(10) generates a cryptographic salt; bcrypt.hash() produces the password hash. The number 10 is the cost factor ($2^{10} = 1024$ hash iterations), balancing security and performance.
4. userService.createUser() inserts into the Users table.
5. userService.assignRole() looks up the 'patient' role UUID and inserts into UserRole. (DELETE + INSERT ensures clean state.)
6. patientService.createPatient() creates the domain profile row in Patient, linked to the new userId.
7. All new self-registrations are automatically given the 'patient' role. Only an Admin can elevate a user to doctor or receptionist.

7.2 Login Flow

1. User submits email + password via POST /api/auth/login.
2. userService.findByEmail() fetches the user record.
3. bcrypt.compare() validates the submitted password against the stored hash. No plaintext password is ever stored or compared directly.
4. userService.getUserRoles() fetches the role name from the RBAC tables.
5. jwt.sign() creates a signed JWT containing { userid, role } with a 7-day expiry.
6. The token is set as an httpOnly cookie (not accessible to JavaScript), preventing XSS token theft.

7.3 Request Authentication Flow

1. Every API request arriving at a protected route passes through getUser(req), which reads the 'token' cookie from the request headers.
2. jwt.verify() validates the token's signature and expiry. An invalid or expired token throws immediately.
3. getUserRoleAndPermissions() hits the database to fetch the user's current role and permission list. This is live DB data — not cached — ensuring that if an admin changes a user's role, the new permissions take effect on the next request.
4. requirePermission(permissionName, access) checks if the fetched permission list contains the required permission string. If not, a "Forbidden" error is thrown and the HTTP response is 403.

7.4 Middleware Route Protection

Next.js middleware (proxy.ts) intercepts every page navigation request (non-API routes) before it reaches any component. The middleware:

- Reads and verifies the JWT from the cookie using the jose library (edge-runtime compatible).
- Redirects authenticated users away from public routes (/login, /register) to their role-specific dashboard.
- Redirects unauthenticated users away from protected routes to /login.
- Prevents cross-role access: a doctor attempting to access /patient/... is redirected to /doctor/dashboard. This is URL-level role isolation.

8. Appointment Lifecycle — State Machine

The appointment is the central operational entity. Its lifecycle is modelled as a strict finite-state machine with deterministic, role-restricted transitions. Each state change is an UPDATE to the status column, validated before execution.

From State	Action	To State	Actor Role	API Endpoint
(none)	Create Appointment	scheduled	Receptionist	POST /api/appointments
scheduled	Check-In	checked_in	Receptionist	PATCH /api/appointments/[id]/checkin
scheduled	Mark No-Show	no_show	Receptionist	PATCH /api/appointments/[id]/no-show
checked_in	Start Consultation	in_progress	Doctor	PATCH /api/appointments/[id]/start
in_progress	Save Clinical Data	completed	Doctor	POST /api/appointments/[id]/interaction
in_progress	Complete	completed	Doctor	PATCH /api/appointments/[id]/complete

Key design decisions:

- A doctor cannot start a consultation unless the receptionist has checked the patient in. The 'checked_in' prerequisite ensures physical presence is confirmed before clinical time starts.
- An appointment becomes 'completed' through two possible paths: either via the /complete endpoint (bare status update) or via the /interaction endpoint (clinical data submission, which internally updates status to 'completed' as part of its transaction). Both paths result in the same terminal state.
- 'no_show' is a terminal state reachable only from 'scheduled'. If a patient has already checked in, they cannot be marked as no-show.
- Every status transition endpoint validates the predecessor state using getAppointmentById() before executing the UPDATE. Attempting an illegal transition returns HTTP 400 with a descriptive error message like 'Invalid transition: cannot start from status scheduled'.

9. Application Flow & Role-Specific Features

9.1 Admin

- Accesses the admin dashboard showing all registered users (getUsersForAdminDashboard — the COALESCE LEFT JOIN query).
- Can create new user accounts (POST /api/admin/users), delete users (DELETE /api/admin/users/[id]), and promote users to doctor/receptionist roles (PUT /api/admin/users/[id]).
- The PUT endpoint creates the corresponding domain profile (doctor, receptionist, or patient row) in addition to assigning the role.
- Can view and create system roles via GET/POST /api/roles.
- Has all 11 permissions (via CROSS JOIN seeding).

9.2 Receptionist

- Views all appointments (GET /api/appointments → INNER JOIN query on appointment + patient + doctor).
- Creates new appointments: searches patients (ILIKE query), searches available doctors (LEFT JOIN availability query), books appointment (conflict check + INSERT).
- Manages appointment status: Check-In (scheduled → checked_in) and No-Show (scheduled → no_show).
- Views all registered patients (GET /api/receptionist/patients → getAllPatients with JOIN to users table).
- Views and updates own profile (GET/PUT /api/receptionist/me).

9.3 Doctor

- Views personalised dashboard with today's appointment count, patients seen, and in-progress consultations (conditional COUNT aggregation with subquery).
- Views their own appointment list (getAppointmentsByDoctorId — filtered SELECT on appointment table).
- Starts a consultation (checked_in → in_progress, updates starttime to NOW()).
- Submits clinical data — the ACID transaction that creates Interaction, batch-inserts Diagnoses and Prescriptions, and completes the appointment atomically.
- Views clinical records for a completed appointment (json_agg aggregation query).

9.4 Patient

- Views personalised dashboard: upcoming appointments, past visits, activity feed (UNION ALL query merging appointment and interaction events).

- Views their own appointments (getAppointmentsByPatientId — filtered SELECT).
- Views their own medical records — reads clinical interaction data for their appointments.
- Has only read_own_data permission. Cannot create or modify any hospital data.
- Self-registration automatically creates a User + Patient + UserRole(patient) in three sequential operations.

10. System Restrictions & How They Are Enforced

Every restriction in the system is traceable to either a database constraint, a trigger, or an application-layer guard. The following catalogue explains each restriction and exactly where it is enforced:

Restriction	Enforcement Layer	Mechanism
No duplicate email addresses	DB + App	UNIQUE(userEmail) in Users; app pre-checks with findByEmail()
One role per user	App	assignRole() DELETES all existing UserRole rows before INSERT
One profile per user	DB	UNIQUE(userId) on Patient, Doctor, Receptionist tables
No double-booking of doctors	App	Conflict SELECT before INSERT; no DB UNIQUE constraint
Appointment state transitions are sequential	App	Status pre-validation before every UPDATE query
Clinical data is all-or-nothing	DB	BEGIN/COMMIT transaction wrapping 4 operations
Patients cannot see other patients' data	App	Ownership check in GET /api/appointments/[id]
Only doctors can submit clinical data	App	requirePermission("clinical.write") at API layer
Only receptionists can check-in patients	App	requirePermission("appointment.update") scoped to role
updatedAt always reflects last change	DB	BEFORE UPDATE trigger on 4 tables
Cascade deletions maintain referential integrity	DB	ON DELETE CASCADE on UserRole, RolePermission, Doctor, Receptionist
Patient records survive account deletion	DB	ON DELETE SET NULL on Patient.userId FK
Role/permission deletions clean up assignments	DB	ON DELETE CASCADE on both FK columns in junction tables
Unauthenticated users cannot access pages	App (Middleware)	JWT verification in proxy.ts; redirect to /login
Users cannot access other roles' pages	App (Middleware)	pathname prefix check in proxy.ts; redirect to own dashboard

11. Database Connection — Pooling Architecture

lib/db.ts — Connection Pool Setup
import { Pool } from 'pg';
const globalForPG = globalThis as unknown as { pool: Pool undefined };
export const pool = globalForPG.pool ?? new Pool({
connectionString: process.env.DATABASE_URL,
ssl: process.env.DATABASE_URL?.includes("localhost")
? false
: { rejectUnauthorized: true },
});
if (!globalForPG.pool) {
globalForPG.pool = pool;
}

Analysis: The application uses node-postgres (pg) connection pooling through a singleton pattern. Key aspects:

- Singleton via globalThis: Next.js hot-module reloading in development mode can re-import modules, which would create a new Pool instance on every reload. By storing the pool on globalThis, subsequent imports reuse the existing pool instead of creating a new one, preventing connection exhaustion.
- SSL configuration: The ternary on DATABASE_URL checks if the connection is to localhost (development), in which case SSL is disabled. For all other environments (production/staging), SSL with rejectUnauthorized: true is enforced, validating the server certificate and preventing MITM attacks.
- Connection string: Using DATABASE_URL (a standard environment variable) makes the application compatible with all major cloud PostgreSQL providers (Supabase, Neon, Railway, AWS RDS).
- pool.connect() test: A connection is attempted at startup and immediately released, providing early failure detection if the database is unreachable.

12. Summary & Conclusion

The Hospital Management System demonstrates a thorough and production-aware implementation of relational database principles within a modern full-stack web application. The DBMS design achieves the following:

- **Structural Integrity:** Thirteen tables in full third normal form, with primary keys, foreign keys, unique constraints, and NOT NULL constraints comprehensively applied.
- **Referential Integrity:** Foreign key constraints with carefully chosen ON DELETE behaviours (CASCADE vs. SET NULL) that match the real-world semantics of each relationship.
- **Access Control:** A four-table RBAC implementation (Users, Role, Permission, UserRole, RolePermission) with seeded permissions that enforce a principle-of-least-privilege model across four user roles.
- **Query Sophistication:** A full range of SQL techniques is employed — INNER JOIN, LEFT JOIN, scalar correlated subqueries, multi-condition joins, json_agg aggregation, UNION ALL, conditional COUNT with FILTER, CASE expressions in ORDER BY, COALESCE for null-coalescing across tables, and dynamic parameterised SQL construction.
- **Transactional Safety:** The most critical write operation (clinical data submission) is wrapped in an explicit BEGIN/COMMIT/ROLLBACK transaction, guaranteeing atomicity across four dependent INSERT/UPDATE operations.
- **Trigger-Based Auditing:** BEFORE UPDATE triggers on all user-editable tables ensure that updatedAt columns are always maintained by the database engine, independent of application code.
- **Authentication Security:** Passwords are hashed using bcrypt with a cost factor of 10. Sessions are managed as httpOnly JWT cookies, preventing XSS token theft. JWT verification happens on every request before any database query.
- **Appointment State Machine:** The appointment lifecycle is modelled as a strict finite-state machine with deterministic transitions validated before every status-changing query, preventing invalid state combinations.

Technical Highlights

- ACID transaction for clinical data writes (interaction.service.ts)
- COALESCE LEFT JOIN across 3 domain tables for unified admin user view (user.service.ts)
- json_agg with DISTINCT and FILTER for clinical record aggregation (interaction.service.ts)
- UNION ALL combining appointment + record events for patient activity feed (dashboard.service.ts)
- Dynamic LEFT JOIN for real-time doctor availability with is_busy flag (doctor.service.ts)
- Conditional COUNT with FILTER for multi-metric dashboard stats in single scan
- Scalar correlated subqueries for userid-to-domainid resolution throughout
- RBAC with cross-join and in-list seeding patterns (definedValues.sql)
- pgcrypto UUID generation — non-sequential, globally unique PKs
- ON DELETE SET NULL vs CASCADE — deliberate per-relationship integrity policy

— *End of Report* —